CS-306-MJ-P Core Java & Web Technology-I

T.Y.B.Sc.(C.S.) Sem-V (NEP 2020) — Practical Slip Solutions (Slips 1–30)

Short, easy-to-study programs • sample outputs from real runs • viva questions for Q3

Slip	Q1 (Java)	Q2 (Web / JS / Node.js)
1	Array sum + ascending order	College registration form (HTML5)
2	Armstrong numbers in range	Student registration form (HTML5)
3	Package: Con & Comp (strings)	College homepage with Flexbox
4	Matrix menu (add/multiply/transpose)	Button changes heading text
5	File reverse + change case	Image gallery (Grid, hover)
6	Account > Saving > AccountDetail	Button/link :hover
7	Driver + user-defined exception	List :first-child / :last-child
8	Abstract Shape + overriding	DOM: replace image on click
9	Employee array, max salary	Arrow functions (+ - * /)
10	Read abc.txt in uppercase	Report with template literals
11	Vehicle inheritance (n vehicles)	Arrow fn: total & average
12	Package game: Indoor/Outdoor	Arrow fn + template literal bill
13	Clock class (validate, AM/PM)	Swap with array destructuring
14	JavaFX prime number	Object destructuring
15	JavaFX calculator	Spread operator array copy
16	JavaFX shopping cart	Rest parameter sum
17	JavaFX key-combination colours	JS module export/import
18	JavaFX colour buttons	Node.js HTTP server
19	Count chars/words/lines of a file	async/await login
20	JavaFX mouse events	Promise divide
21	College > Department	Node.js path module
22	Product + interface + count	fs async operations
23	Multilevel: Continent>Country>State	fs create/write/read/append
24	Abstract Shape > Cylinder	Sync vs async fs
25	Interface Calculator	Directories + JSON
26	Student attendance exception	fs copy/rename/delete
27	Person (5 objects, this)	Read, modify, write JSON
28	MyDate + InvalidDateException	List files by extension
29	Number 0 exception + static prime	fs.stat()
30	MyNumber + command line args	Node.js file manager (menu)

How to run

Java: `javac File.java` then `java File` (Java 8 or later).
JavaFX (Slips 14-18, 20): JavaFX SDK must be installed; run with `--module-path <sdk>/lib --add-modules javafx.controls` (or use NetBeans/Eclipse with JavaFX).
JavaScript / Node.js: install Node.js, then `node file.js`. Slips 9-16, 19-20 also run in the browser console.
HTML/CSS: save as `.html` and open in Chrome/Edge.

Slip 1

Q1 [15 marks]

Write a Java program to print the sum of elements of the array. Also display array elements in ascending order.

ArraySum.java

```java
import java.util.*;

public class ArraySum {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("How many elements? ");
        int n = sc.nextInt(), sum = 0;
        int[] arr = new int[n];
        System.out.println("Enter " + n + " elements:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
            sum += arr[i];                      // add each element
        }
        System.out.println("Sum = " + sum);
        Arrays.sort(arr);                       // ascending order
        System.out.println("Ascending order: " + Arrays.toString(arr));
    }
}
```

Run: `javac ArraySum.java`
`java ArraySum`

Output

```
How many elements? 5
Enter 5 elements:
40 10 30 20 50
Sum = 150
Ascending order: [10, 20, 30, 40, 50]
```

Q2 [15 marks]

Design a College/School Registration Form using HTML5 and CSS3 and apply appropriate HTML5 validation attributes.

college_registration.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>College Registration Form</title>
<style>
  body  { font-family: Arial, sans-serif; background: #eef2f7; margin: 0; padding: 20px; }
  form  { max-width: 500px; margin: auto; background: #fff; padding: 25px; border-radius: 8px;
          box-shadow: 0 2px 8px rgba(0,0,0,.2); }
  h2    { text-align: center; color: #1a4d8f; }
  label { display: block; margin-top: 12px; font-weight: bold; }
  input[type=text], input[type=email], input[type=tel], input[type=password],
  input[type=number], input[type=date], select, textarea
        { width: 100%; padding: 8px; margin-top: 4px; box-sizing: border-box; }
  input:focus:invalid { border: 2px solid red; }      /* red while typing wrong value */
  input:focus:valid   { border: 2px solid green; }    /* green when value is valid */
  button { margin-top: 18px; width: 100%; padding: 10px; background: #1a4d8f; color: #fff;
           border: none; border-radius: 4px; font-size: 16px; cursor: pointer; }
  button:hover { background: #0f3563; }
</style>
</head>
<body>
<form action="#" method="post">
  <h2>College Registration Form</h2>

  <label for="name">Full Name</label>
  <input type="text" id="name" name="name" required minlength="3" maxlength="40"
         pattern="[A-Za-z ]+" title="Only letters and spaces" placeholder="Enter your name">

  <label for="email">Email</label>
  <input type="email" id="email" name="email" required placeholder="you@example.com">

  <label for="mobile">Mobile Number</label>
  <input type="tel" id="mobile" name="mobile" required pattern="[6-9][0-9]{9}"
         title="10 digit number starting with 6-9">

  <label for="dob">Date of Birth</label>
  <input type="date" id="dob" name="dob" required max="2008-12-31">

  <label>Gender</label>
  <input type="radio" name="gender" value="M" required> Male
  <input type="radio" name="gender" value="F"> Female
```

```
  <label for="course">Course</label>
  <select id="course" name="course" required>
    <option value="">-- Select --</option>
    <option>B.Sc. Computer Science</option>
    <option>B.Sc. Electronics</option>
    <option>B.C.A.</option>
  </select>

  <label for="percent">12th Percentage</label>
  <input type="number" id="percent" name="percent" min="35" max="100" step="0.01" required>

  <label for="address">Address</label>
  <textarea id="address" name="address" rows="3" required></textarea>

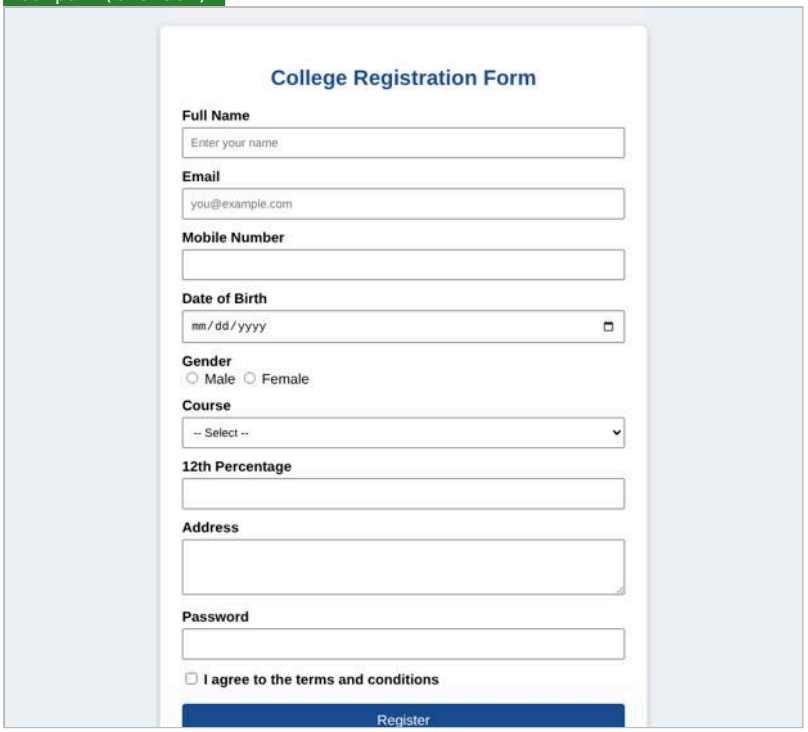
  <label for="pass">Password</label>
  <input type="password" id="pass" name="pass" required minlength="8">

  <label><input type="checkbox" required> I agree to the terms and conditions</label>

  <button type="submit">Register</button>
</form>
</body>
</html>
```

Run: save as `college_registration.html` and open it in the browser.

Output (browser)



Q3 Viva [5 marks]

- **What is an array?** A fixed-size collection of same-type elements, index starts from 0; length gives its size.
- **Which method sorts an array?** Arrays.sort(arr) from java.util (ascending order).
- **Name some HTML5 validation attributes.** required, pattern, minlength/maxlength, min/max, type=email/tel/number/date.

Slip 2

Q1 [15 marks]

Write a Java Program to Display Armstrong Numbers Between range. Accept range from user.

Armstrong.java

```java
import java.util.*;

public class Armstrong {
    // Armstrong: sum of (digit ^ number of digits) == number
    static boolean isArmstrong(int num) {
        int digits = String.valueOf(num).length(), temp = num, sum = 0;
        while (temp > 0) {
            sum += Math.pow(temp % 10, digits);
            temp /= 10;
        }
        return sum == num;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter start and end of range: ");
        int start = sc.nextInt(), end = sc.nextInt();
        System.out.println("Armstrong numbers between " + start + " and " + end + ":");
        for (int i = start; i <= end; i++)
            if (isArmstrong(i)) System.out.print(i + " ");
        System.out.println();
    }
}
```

Run: `javac Armstrong.java`
`java Armstrong`

Output

```
Enter start and end of range: 100 1000
Armstrong numbers between 100 and 1000:
153 370 371 407
```

Q2 [15 marks]

Design a Student Registration Form using HTML5 and CSS3 and apply appropriate HTML5 validation attributes.

student_registration.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Student Registration Form</title>
<style>
  body { font-family: Verdana, sans-serif; background: #f4efe6; padding: 20px; }
  .box { max-width: 480px; margin: auto; background: #fff; padding: 20px 25px;
         border-top: 6px solid #c0392b; border-radius: 6px; }
  h2   { text-align: center; color: #c0392b; }
  .row { margin-bottom: 14px; }
  .row label { display: block; margin-bottom: 4px; }
  input:not([type=radio]):not([type=checkbox]), select { width: 100%; padding: 7px; box-sizing: border-box; }
  input:focus, select:focus { outline: 2px solid #c0392b; }
  input[type=submit], input[type=reset] { width: 48%; background: #c0392b; color: #fff;
         border: 0; padding: 10px; cursor: pointer; }
</style>
</head>
<body>
<div class="box">
  <h2>Student Registration</h2>
  <form action="#" method="post">
    <div class="row"><label>Roll Number</label>
      <input type="number" name="roll" min="1" max="9999" required></div>

    <div class="row"><label>Student Name</label>
      <input type="text" name="sname" required pattern="[A-Za-z ]{3,30}" title="3 to 30 letters"></div>

    <div class="row"><label>Email</label>
      <input type="email" name="email" required></div>

    <div class="row"><label>Phone</label>
      <input type="tel" name="phone" pattern="[0-9]{10}" required title="10 digits"></div>

    <div class="row"><label>Class</label>
      <select name="class" required>
        <option value="">Select class</option>
        <option>F.Y.B.Sc.</option><option>S.Y.B.Sc.</option><option>T.Y.B.Sc.</option>
      </select></div>
```

```
    <div class="row"><label>Gender</label>
      <input type="radio" name="g" required> Male
      <input type="radio" name="g"> Female</div>

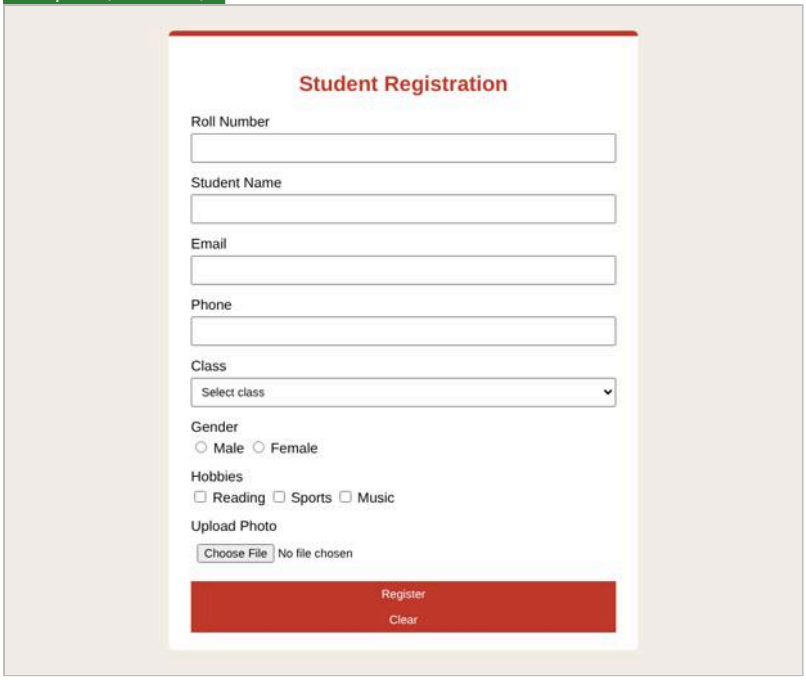
    <div class="row"><label>Hobbies</label>
      <input type="checkbox"> Reading
      <input type="checkbox"> Sports
      <input type="checkbox"> Music</div>

    <div class="row"><label>Upload Photo</label>
      <input type="file" accept="image/*" required></div>

    <input type="submit" value="Register"> <input type="reset" value="Clear">
  </form>
</div>
</body>
</html>
```

Run: save as `student_registration.html` and open it in the browser.

Output (browser)



Q3 Viva [5 marks]

- **What is an Armstrong number?** A number equal to the sum of its digits each raised to the power of number of digits, e.g. 153 = 1^3+5^3+3^3.
- **Why Scanner?** Scanner (java.util) reads input from keyboard: nextInt(), nextLine() etc.
- **Difference between placeholder and value?** placeholder is a hint that disappears on typing; value is actual default content.

Slip 3

Q1 [15 marks]

Write a package for String operation which has two classes Con and Comp. Con class has to concatenate two strings and comp class compares two strings. Also display proper messages on execution.

strops/Comp.java

```java
package strops;

public class Comp {
    public void compare(String a, String b) {
        if (a.equals(b))
            System.out.println("Both strings are equal");
        else
            System.out.println("Strings are NOT equal (compareTo = " + a.compareTo(b) + ")");
    }
}
```

strops/Con.java

```java
package strops;

public class Con {
    public void concat(String a, String b) {
        System.out.println("Concatenation of \"" + a + "\" and \"" + b + "\" = " + a.concat(b));
    }
}
```

StringDemo.java

```java
import strops.Con;
import strops.Comp;
import java.util.*;

public class StringDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first string : ");
        String s1 = sc.nextLine();
        System.out.print("Enter second string: ");
        String s2 = sc.nextLine();
        new Con().concat(s1, s2);
        new Comp().compare(s1, s2);
    }
}
```

Run: `javac strops/*.java StringDemo.java`
`java StringDemo`

Output

```
Enter first string : Hello
Enter second string: World
Concatenation of "Hello" and "World" = HelloWorld
Strings are NOT equal (compareTo = -15)
```

Q2 [15 marks]

Create a responsive webpage using CSS Flexbox for a college homepage containing header, navigation, content and footer. Use appropriate Flexbox alignment and distribution properties

college_flexbox.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>College Homepage (Flexbox)</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; font-family: Arial, sans-serif; }
  body { display: flex; flex-direction: column; min-height: 100vh; }   /* footer stays at bottom */

  header { display: flex; justify-content: space-between; align-items: center;
           background: #1a4d8f; color: #fff; padding: 15px 25px; }

  nav { display: flex; justify-content: center; flex-wrap: wrap; gap: 10px; background: #0f3563; }
  nav a { color: #fff; text-decoration: none; padding: 12px 18px; }
  nav a:hover { background: #1a4d8f; }

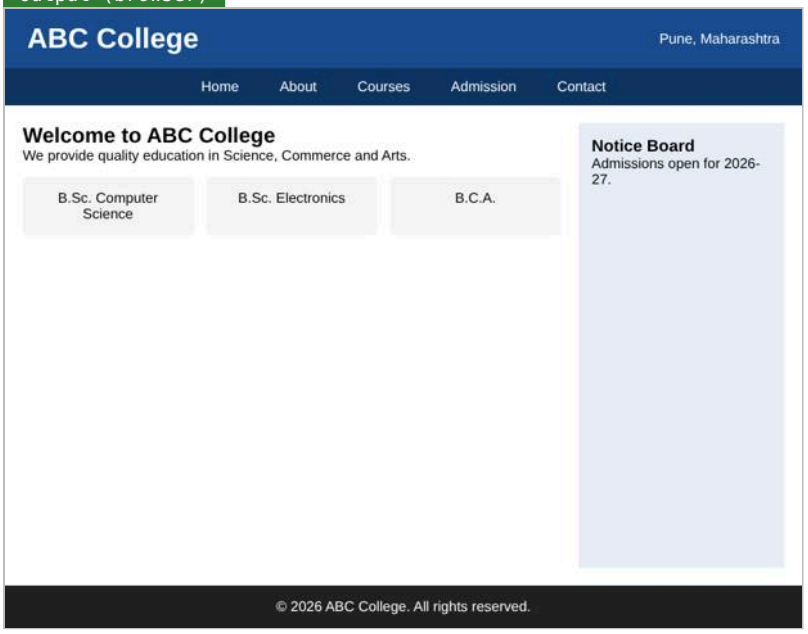
  main { display: flex; flex: 1; gap: 20px; padding: 20px; }          /* content grows */
  .content { flex: 3; }
  .side    { flex: 1; background: #e8eef7; padding: 15px; }
  .cards   { display: flex; justify-content: space-between; gap: 15px; margin-top: 15px; }
  .card    { flex: 1; background: #f5f5f5; padding: 15px; border-radius: 6px; text-align: center; }
```

```
  footer { display: flex; justify-content: center; align-items: center;
           background: #222; color: #fff; padding: 15px; }

  @media (max-width: 700px) {            /* responsive: stack on small screens */
    main, .cards, header { flex-direction: column; text-align: center; gap: 10px; }
  }
</style>
</head>
<body>
  <header><h1>ABC College</h1><p>Pune, Maharashtra</p></header>
  <nav><a href="#">Home</a><a href="#">About</a><a href="#">Courses</a><a href="#">Admission</a><a href="#">Contact</a></nav>
  <main>
    <section class="content">
      <h2>Welcome to ABC College</h2>
      <p>We provide quality education in Science, Commerce and Arts.</p>
      <div class="cards">
        <div class="card">B.Sc. Computer Science</div>
        <div class="card">B.Sc. Electronics</div>
        <div class="card">B.C.A.</div>
      </div>
    </section>
    <aside class="side"><h3>Notice Board</h3><p>Admissions open for 2026-27.</p></aside>
  </main>
  <footer>&copy; 2026 ABC College. All rights reserved.</footer>
</body>
</html>
```

Run: save as `college_flexbox.html` and open it in the browser.

Output (browser)



Q3 Viva [5 marks]

- **What is a package?** A namespace/folder that groups related classes; declared with the package keyword as first statement and used by import.
- **equals() vs ==?** equals() compares string content; == compares references.
- **Flexbox: justify-content vs align-items?** justify-content aligns along the main axis, align-items along the cross axis.

Slip 4

Q1 [15 marks]

Write a menu driven program to perform the following operations on multidimensional array ie matrix : i. Addition ii. Multiplication iii. Transpose of any matrix. iv. Exit

MatrixMenu.java

```java
import java.util.*;

public class MatrixMenu {
    static Scanner sc = new Scanner(System.in);

    static int[][] read(int r, int c) {
        int[][] m = new int[r][c];
        System.out.println("Enter " + r + "x" + c + " matrix:");
        for (int i = 0; i < r; i++)
            for (int j = 0; j < c; j++) m[i][j] = sc.nextInt();
        return m;
    }

    static void show(int[][] m) {
        for (int[] row : m) System.out.println(Arrays.toString(row));
    }

    public static void main(String[] args) {
        int ch;
        do {
            System.out.println("\n1.Addition 2.Multiplication 3.Transpose 4.Exit");
            System.out.print("Choice: ");
            ch = sc.nextInt();
            if (ch == 1) {
                System.out.print("Rows and columns: ");
                int r = sc.nextInt(), c = sc.nextInt();
                int[][] a = read(r, c), b = read(r, c), s = new int[r][c];
                for (int i = 0; i < r; i++)
                    for (int j = 0; j < c; j++) s[i][j] = a[i][j] + b[i][j];
                System.out.println("Sum:"); show(s);
            } else if (ch == 2) {
                System.out.print("Order of A (r1 c1) and columns of B (c2): ");
                int r1 = sc.nextInt(), c1 = sc.nextInt(), c2 = sc.nextInt();
                int[][] a = read(r1, c1), b = read(c1, c2), p = new int[r1][c2];
                for (int i = 0; i < r1; i++)
                    for (int j = 0; j < c2; j++)
                        for (int k = 0; k < c1; k++) p[i][j] += a[i][k] * b[k][j];
                System.out.println("Product:"); show(p);
            } else if (ch == 3) {
                System.out.print("Rows and columns: ");
                int r = sc.nextInt(), c = sc.nextInt();
                int[][] a = read(r, c), t = new int[c][r];
                for (int i = 0; i < r; i++)
                    for (int j = 0; j < c; j++) t[j][i] = a[i][j];
                System.out.println("Transpose:"); show(t);
            }
        } while (ch != 4);
        System.out.println("Bye!");
    }
}
```

Run: `javac MatrixMenu.java`
`java MatrixMenu`

Output

```
1.Addition 2.Multiplication 3.Transpose 4.Exit
Choice: 1
Rows and columns: 2 2
Enter 2x2 matrix:
1 2
3 4
Enter 2x2 matrix:
5 6
7 8
Sum:
[6, 8]
[10, 12]

1.Addition 2.Multiplication 3.Transpose 4.Exit
Choice: 2
Order of A (r1 c1) and columns of B (c2): 2 3 2
Enter 2x3 matrix:
1 2 3
4 5 6
Enter 3x2 matrix:
7 8
9 10
```

```
11 12
Product:
[58, 64]
[139, 154]

1.Addition 2.Multiplication 3.Transpose 4.Exit
Choice: 3
Rows and columns: 2 3
Enter 2x3 matrix:
1 2 3
4 5 6
Transpose:
[1, 4]
[2, 5]
[3, 6]

1.Addition 2.Multiplication 3.Transpose 4.Exit
Choice: 4
Bye!
```

Q2 [15 marks]

Create a webpage containing a heading and a button. When the button is clicked, change the heading text from "Hello! Welcome" to "Text Changed Successfully!".

change_heading.html

```html
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Change Heading</title>
</head>
<body style="text-align:center; font-family:Arial; margin-top:60px;">
  <h1 id="heading">Hello! Welcome</h1>
  <button id="btn">Change Text</button>

  <script>
    document.getElementById("btn").onclick = function () {
      document.getElementById("heading").innerText = "Text Changed Successfully!";
    };
  </script>
</body>
</html>
```

Run: save as `change_heading.html` and open it in the browser.

Output (browser)

Page shows heading "Hello! Welcome". After clicking the button the heading becomes "Text Changed Successfully!" (verified in browser).

Q3 Viva [5 marks]

- **When can two matrices be multiplied?** When columns of first = rows of second; result is rows1 x cols2.
- **What is transpose?** Rows become columns: t[j][i] = a[i][j].
- **innerText vs innerHTML?** innerText sets plain text; innerHTML parses HTML tags.

Slip 5

Q1 [15 marks]

Write a program to accept a text file from user and display the contents of a file in reverse order and change its case.

ReverseCase.java

```java
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class ReverseCase {
    public static void main(String[] args) throws IOException {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter file name: ");
        String text = Files.readString(Path.of(sc.nextLine()));

        String reversed = new StringBuilder(text).reverse().toString();   // reverse order
        StringBuilder out = new StringBuilder();
        for (char ch : reversed.toCharArray()) {                          // change case
            if (Character.isUpperCase(ch)) out.append(Character.toLowerCase(ch));
            else out.append(Character.toUpperCase(ch));
        }
        System.out.println("Original :\n" + text);
        System.out.println("Reversed + case changed :\n" + out);
    }
}
```

Run: `javac ReverseCase.java`
`java ReverseCase` Create **notes.txt** in the same folder first (any text).

Output

```
Enter file name: notes.txt
Original :
Hello World
Java Is Fun

Reversed + case changed :

NUf Si AVAj
DLROw OLLEh
```

Q2 [15 marks]

Create an image gallery using HTML5 and CSS3 with hover effects, responsive images and Flexbox/Grid layout.

gallery.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Image Gallery</title>
<style>
  body { font-family: Arial, sans-serif; background: #111; color: #fff; margin: 0; padding: 20px; }
  h1   { text-align: center; }
  .gallery { display: grid; gap: 15px;                                    /* Grid layout */
             grid-template-columns: repeat(auto-fit, minmax(220px, 1fr)); }
  .item { position: relative; overflow: hidden; border-radius: 8px; }
  .item img { width: 100%; height: 100%; display: block; object-fit: cover;  /* responsive image */
              transition: transform .4s; }
  .item:hover img { transform: scale(1.15); }                             /* hover: zoom */
  .item span { position: absolute; bottom: 0; left: 0; right: 0; padding: 8px; text-align: center;
               background: rgba(0,0,0,.6); opacity: 0; transition: opacity .4s; }
  .item:hover span { opacity: 1; }                                        /* hover: caption */
</style>
</head>
<body>
  <h1>My Gallery</h1>
  <div class="gallery">
    <div class="item"><img src="https://picsum.photos/id/10/400/300" alt="Forest"><span>Forest</span></div>
    <div class="item"><img src="https://picsum.photos/id/20/400/300" alt="Desk"><span>Desk</span></div>
    <div class="item"><img src="https://picsum.photos/id/29/400/300" alt="Mountain"><span>Mountain</span></div>
    <div class="item"><img src="https://picsum.photos/id/37/400/300" alt="Lake"><span>Lake</span></div>
    <div class="item"><img src="https://picsum.photos/id/42/400/300" alt="Coffee"><span>Coffee</span></div>
    <div class="item"><img src="https://picsum.photos/id/57/400/300" alt="City"><span>City</span></div>
  </div>
</body>
</html>
```

Run: save as `gallery.html` and open it in the browser.

Output (browser)

Responsive grid of 6 photos; on hover the image zooms and a caption slides in. (Images load from picsum.photos - replace

with your own image files if offline.)

Q3 Viva [5 marks]

- **How do you read a file in Java?** FileReader/BufferedReader or Files.readString(Path).
- **Flexbox vs Grid?** Flexbox is one-dimensional (row or column); Grid is two-dimensional (rows and columns).
- **What does object-fit: cover do?** Scales the image to fill its box while keeping the aspect ratio (crops extra).

Slip 6

Q1 [15 marks]

Write a program to define a class Account having members custname, accno. Define default and parameterized constructor. Create a subclass called SavingAccount with members savingbal, minbal. Create a derived class AccountDetail that extends the class SavingAccount with members, depositamt and withdrawalamt. Write a appropriate method to display customer details.

AccountDemo.java

```java
class Account {
    String custname; int accno;
    Account() { custname = "Unknown"; accno = 0; }                // default constructor
    Account(String n, int a) { custname = n; accno = a; }         // parameterized
}

class SavingAccount extends Account {
    double savingbal, minbal;
    SavingAccount(String n, int a, double bal, double min) {
        super(n, a);
        savingbal = bal; minbal = min;
    }
}

class AccountDetail extends SavingAccount {
    double depositamt, withdrawalamt;
    AccountDetail(String n, int a, double bal, double min, double dep, double wd) {
        super(n, a, bal, min);
        depositamt = dep; withdrawalamt = wd;
    }
    void display() {
        System.out.println("Customer Name : " + custname);
        System.out.println("Account No    : " + accno);
        System.out.println("Saving Balance: " + savingbal);
        System.out.println("Minimum Bal   : " + minbal);
        System.out.println("Deposit       : " + depositamt);
        System.out.println("Withdrawal    : " + withdrawalamt);
        System.out.println("Current Bal   : " + (savingbal + depositamt - withdrawalamt));
    }
}

public class AccountDemo {
    public static void main(String[] args) {
        AccountDetail d = new AccountDetail("Rudra Patil", 1001, 5000, 1000, 2000, 1500);
        d.display();
    }
}
```

Run: `javac AccountDemo.java`
`java AccountDemo`

Output

```
Customer Name : Rudra Patil
Account No    : 1001
Saving Balance: 5000.0
Minimum Bal   : 1000.0
Deposit       : 2000.0
Withdrawal    : 1500.0
Current Bal   : 5500.0
```

Q2 [15 marks]

Create a webpage containing a button and link. Change their appearance when the mouse pointer moves over them. (Use :hover Pseudo-Class)

hover.html

```html
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>:hover Pseudo-class</title>
<style>
  body { text-align: center; font-family: Arial; margin-top: 60px; }

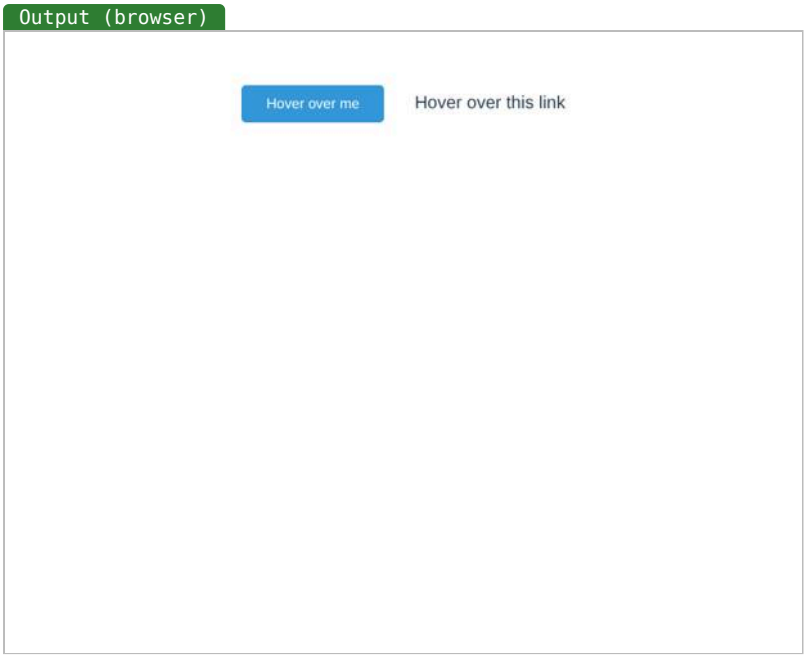
  button { padding: 12px 28px; font-size: 16px; background: #3498db; color: #fff;
           border: none; border-radius: 5px; cursor: pointer; transition: .3s; }
  button:hover { background: #e74c3c; transform: scale(1.1); }    /* button on hover */

  a { font-size: 20px; color: #2c3e50; text-decoration: none; margin-left: 30px; }
  a:hover { color: #fff; background: #27ae60; text-decoration: underline; padding: 6px; } /* link on hover */
</style>
</head>
<body>
  <button>Hover over me</button>
```

```
  <a href="https://www.google.com">Hover over this link</a>
</body>
</html>
```

Run: save as `hover.html` and open it in the browser.

Output (browser)



Q3 Viva [5 marks]

- **Types of inheritance?** Single, multilevel, hierarchical (classes); multiple inheritance only through interfaces.
- **Use of super?** Calls the parent constructor/method: super(...) must be the first statement.
- **Pseudo-class vs pseudo-element?** Pseudo-class (:hover) selects a state; pseudo-element (::before) styles a part of an element.

Slip 7

Q1 [15 marks]

Write a class Driver with attributes license_no, name, address and age. Initialize values through the parameterized constructor. If age of Driver is less than 18 then user-defined exception should be generated —Age is below 18 years—

DriverDemo.java

```
import java.util.*;

class AgeBelow18Exception extends Exception {            // user-defined exception
    AgeBelow18Exception(String msg) { super(msg); }
}

class Driver {
    int license_no, age; String name, address;
    Driver(int l, String n, String ad, int a) throws AgeBelow18Exception {
        if (a < 18) throw new AgeBelow18Exception("Age is below 18 years");
        license_no = l; name = n; address = ad; age = a;
    }
    void display() {
        System.out.println("License: " + license_no + ", Name: " + name +
                           ", Address: " + address + ", Age: " + age);
    }
}

public class DriverDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("License no, name, address, age: ");
        int l = sc.nextInt(); String n = sc.next(), ad = sc.next(); int a = sc.nextInt();
        try {
            new Driver(l, n, ad, a).display();
        } catch (AgeBelow18Exception e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

Run: `javac DriverDemo.java`
`java DriverDemo`

Output

```
License no, name, address, age: 101 Amit Pune 16
Exception: Age is below 18 years
```

Q2 [15 marks]

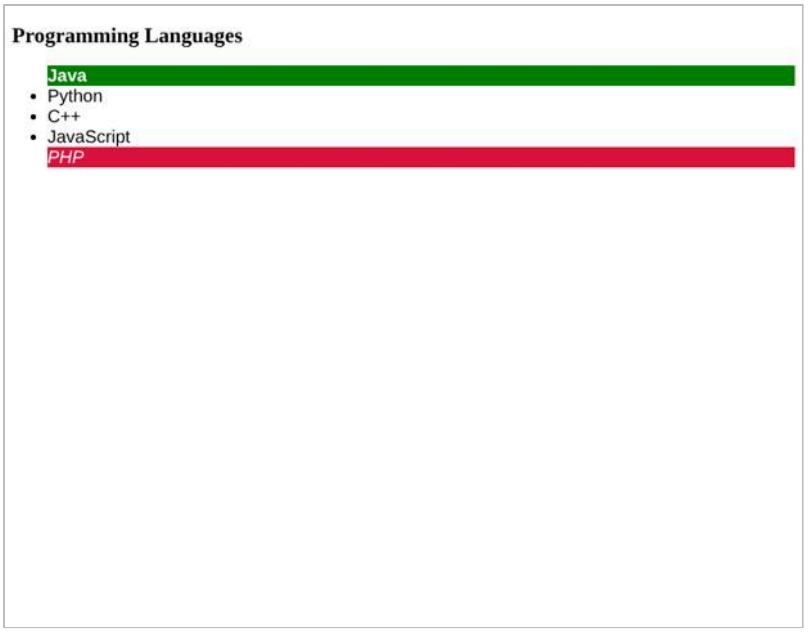
Create an unordered list of Programming Language names and apply different styles to the first and last list items.(Use :first-child and :last- child)

list_styles.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>first-child and last-child</title>
<style>
  ul { font-size: 20px; font-family: Arial; }
  li:first-child { color: white; background: green; font-weight: bold; }     /* first item */
  li:last-child  { color: white; background: crimson; font-style: italic; }  /* last item */
</style>
</head>
<body>
  <h2>Programming Languages</h2>
  <ul>
    <li>Java</li>
    <li>Python</li>
    <li>C++</li>
    <li>JavaScript</li>
    <li>PHP</li>
  </ul>
</body>
</html>
```

Run: save as `list_styles.html` and open it in the browser.

Output (browser)



Q3 Viva [5 marks]

- **How to create a user-defined exception?** Extend the Exception class and pass the message with super(msg).
- **throw vs throws?** throw raises an exception object; throws declares exceptions a method may raise.
- **What do :first-child and :last-child select?** The first / last child element of its parent.

Slip 8

Q1 [15 marks]

Write a program to create an abstract class named Shape that contains two integers and an empty method named printArea(). Provide three classes named Rectangle, Triangle and Circle such that each one of the classes extends the class Shape. Each one of the classes contain only the method printArea() that prints the area of the given shape. (use method overriding).

ShapeDemo.java

```java
abstract class Shape {
    int a, b;
    Shape(int a, int b) { this.a = a; this.b = b; }
    abstract void printArea();                  // overridden by subclasses
}

class Rectangle extends Shape {
    Rectangle(int l, int w) { super(l, w); }
    void printArea() { System.out.println("Area of Rectangle = " + (a * b)); }
}

class Triangle extends Shape {
    Triangle(int b, int h) { super(b, h); }
    void printArea() { System.out.println("Area of Triangle = " + (0.5 * a * b)); }
}

class Circle extends Shape {
    Circle(int r) { super(r, 0); }
    void printArea() { System.out.println("Area of Circle = " + (Math.PI * a * a)); }
}

public class ShapeDemo {
    public static void main(String[] args) {
        Shape[] s = { new Rectangle(10, 5), new Triangle(6, 4), new Circle(7) };
        for (Shape x : s) x.printArea();        // runtime polymorphism
    }
}
```

Run: `javac ShapeDemo.java`
`java ShapeDemo`

Output

```
Area of Rectangle = 50
Area of Triangle = 12.0
Area of Circle = 153.93804002589985
```

Q2 [15 marks]

Create a webpage using JavaScript DOM manipulation and event handling with an image and a button. When the button is clicked, replace the displayed image with another image.

replace_image.html

```html
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Replace Image</title>
</head>
<body style="text-align:center; font-family:Arial;">
  <h2>Click the button to change the image</h2>
  <img id="pic" src="https://picsum.photos/id/10/400/250" width="400" alt="picture"><br><br>
  <button id="btn">Replace Image</button>

  <script>
    const img = document.getElementById("pic");
    const btn = document.getElementById("btn");
    const first = "https://picsum.photos/id/10/400/250";
    const second = "https://picsum.photos/id/29/400/250";

    btn.addEventListener("click", () => {          // event handling
      img.src = img.src.includes("id/10") ? second : first;   // DOM manipulation
    });
  </script>
</body>
</html>
```

Run: save as `replace_image.html` and open it in the browser.

Output (browser)

Shows an image and a button; every click swaps the image between the two URLs (verified in browser). Replace the URLs with your own image files if offline.

Q3 Viva [5 marks]

- **Abstract class?** A class declared abstract that cannot be instantiated; may contain abstract methods without body.
- **Overriding vs overloading?** Overriding: same signature in subclass (runtime). Overloading: same name, different parameters (compile time).
- **What is the DOM?** Document Object Model - tree representation of the page that JavaScript can read and change.

Slip 9

Q1 [15 marks]

Write a program which define class Employee with data member as id, name and salary Store the information of 'n' employees and display the name of employee having maximum salary (Use array of object)

EmployeeMax.java

```java
import java.util.*;

class Employee {
    int id; String name; double salary;
    Employee(int id, String name, double salary) {
        this.id = id; this.name = name; this.salary = salary;
    }
}

public class EmployeeMax {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Number of employees: ");
        int n = sc.nextInt();
        Employee[] e = new Employee[n];                    // array of objects
        for (int i = 0; i < n; i++) {
            System.out.print("Employee " + (i + 1) + " (id name salary): ");
            e[i] = new Employee(sc.nextInt(), sc.next(), sc.nextDouble());
        }
        Employee max = e[0];
        for (Employee x : e) if (x.salary > max.salary) max = x;
        System.out.println("Highest salary: " + max.name + " (" + max.salary + ")");
    }
}
```

Run: `javac EmployeeMax.java`
`java EmployeeMax`

Output

```
Number of employees: 3
Employee 1 (id name salary): 1 Amit 30000
Employee 2 (id name salary): 2 Sara 52000
Employee 3 (id name salary): 3 Ravi 41000
Highest salary: Sara (52000.0)
```

Q2 [15 marks]

Write a JavaScript program to demonstrate Arrow Functions for addition, subtraction, multiplication and division.

arrow_ops.js

```javascript
// Arrow functions: (parameters) => expression
const add      = (a, b) => a + b;
const subtract = (a, b) => a - b;
const multiply = (a, b) => a * b;
const divide   = (a, b) => (b === 0 ? "Cannot divide by zero" : a / b);

const x = 20, y = 5;
console.log("Addition       :", add(x, y));
console.log("Subtraction    :", subtract(x, y));
console.log("Multiplication :", multiply(x, y));
console.log("Division       :", divide(x, y));
console.log("Division by 0  :", divide(x, 0));
```

Run: `node arrow_ops.js`

Output

```
Addition       : 25
Subtraction    : 15
Multiplication : 100
Division       : 4
Division by 0  : Cannot divide by zero
```

Q3 Viva [5 marks]

- **Array of objects?** Each element must be created with new; the array only holds references.
- **Arrow function syntax?** (a, b) => a + b; shorter than function and has no own this.
- **Why store maximum with a loop?** Compare each salary with the current max and update when larger.

Slip 10

Q1 [15 marks]

Write a program to read the contents of "abc.txt" file. Display the contents of file in uppercase as output.

FileUpper.java

```java
import java.io.*;

public class FileUpper {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new FileReader("abc.txt"))) {
            String line;
            while ((line = br.readLine()) != null)
                System.out.println(line.toUpperCase());
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Run: `javac FileUpper.java`
`java FileUpper` Create **abc.txt** in the same folder first.

Output

```
HELLO JAVA
THIS IS ABC FILE
```

Q2 [15 marks]

Write a JavaScript program to store student information and marks in variables and generate a formatted student report using template literals

student_report.js

```javascript
// Student information stored in variables
const name = "Riya Patil";
const rollNo = 21;
const className = "T.Y.B.Sc.(C.S.)";
const maths = 78, java = 85, web = 91;
const total = maths + java + web;
const percentage = (total / 3).toFixed(2);

// Template literal: back-ticks ` ` and ${ } placeholders, multi-line allowed
const report = `
===== STUDENT REPORT =====
Name       : ${name}
Roll No    : ${rollNo}
Class      : ${className}
Maths      : ${maths}
Java       : ${java}
Web Tech   : ${web}
Total      : ${total}
Percentage : ${percentage}%
Result     : ${percentage >= 40 ? "PASS" : "FAIL"}
==========================`;
console.log(report);
```

Run: `node student_report.js`

Output

```
===== STUDENT REPORT =====
Name       : Riya Patil
Roll No    : 21
Class      : T.Y.B.Sc.(C.S.)
Maths      : 78
Java       : 85
Web Tech   : 91
Total      : 254
Percentage : 84.67%
Result     : PASS
==========================
```

Q3 Viva [5 marks]

- **Which classes read a text file?** FileReader with BufferedReader (readLine()).
- **What is try-with-resources?** Automatically closes the resource (file) at the end of the try block.
- **What are template literals?** Strings written in back-ticks supporting ${expression} and multi-line text.

Slip 11

Q1 [15 marks]

Write a program to create a super class Vehicle having members Company and price. Derive two different classes LightMotorVehicle (mileage) and HeavyMotorVehicle (capacity_in_tons). Accept the information for "n" vehicles and display the information in appropriate form. While taking data, ask user about the type of vehicle first.

VehicleDemo.java

```java
import java.util.*;

class Vehicle {
    String company; double price;
    Vehicle(String c, double p) { company = c; price = p; }
    void display() { System.out.print("Company: " + company + ", Price: " + price); }
}

class LightMotorVehicle extends Vehicle {
    double mileage;
    LightMotorVehicle(String c, double p, double m) { super(c, p); mileage = m; }
    void display() { super.display(); System.out.println(", Mileage: " + mileage + " km/l"); }
}

class HeavyMotorVehicle extends Vehicle {
    double capacity;
    HeavyMotorVehicle(String c, double p, double t) { super(c, p); capacity = t; }
    void display() { super.display(); System.out.println(", Capacity: " + capacity + " tons"); }
}

public class VehicleDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("How many vehicles? ");
        int n = sc.nextInt();
        Vehicle[] v = new Vehicle[n];
        for (int i = 0; i < n; i++) {
            System.out.print("Type (1=Light, 2=Heavy): ");
            int type = sc.nextInt();
            System.out.print("Company and price: ");
            String c = sc.next(); double p = sc.nextDouble();
            if (type == 1) {
                System.out.print("Mileage: ");
                v[i] = new LightMotorVehicle(c, p, sc.nextDouble());
            } else {
                System.out.print("Capacity in tons: ");
                v[i] = new HeavyMotorVehicle(c, p, sc.nextDouble());
            }
        }
        System.out.println("\n--- Vehicle details ---");
        for (Vehicle x : v) x.display();
    }
}
```

Run: `javac VehicleDemo.java`
`java VehicleDemo` Enter each value on its own line as shown.

Output

```
How many vehicles? 2
Type (1=Light, 2=Heavy): 1
Company and price: Honda 90000
Mileage: 55
Type (1=Light, 2=Heavy): 2
Company and price: Tata 2500000
Capacity in tons: 12.5

--- Vehicle details ---
Company: Honda, Price: 90000.0, Mileage: 55.0 km/l
Company: Tata, Price: 2500000.0, Capacity: 12.5 tons
```

Q2 [15 marks]

Write a JavaScript program using an arrow function to calculate the total and average marks of three subjects. Display the result using template literals

total_average.js

```javascript
// Arrow function returning total and average
const result = (m1, m2, m3) => {
  const total = m1 + m2 + m3;
  return { total, average: total / 3 };
};

const { total, average } = result(70, 85, 90);

console.log(`Marks   : 70, 85, 90`);
```

```
console.log(`Total   : ${total}`);
console.log(`Average : ${average.toFixed(2)}`);
```

Run: `node total_average.js`

Output

```
Marks   : 70, 85, 90
Total   : 245
Average : 81.67
```

Q3 Viva [5 marks]

- **What is method overriding here?** display() in the subclasses overrides the one in Vehicle and calls super.display().
- **Why ask the vehicle type first?** To decide which subclass object (Light or Heavy) must be created.
- **Why return an object from an arrow function?** To send back several values (total and average) which can be destructured.

Slip 12

Q1 [15 marks]

Write a package game which will have 2 classes Indoor & Outdoor. Use a function display () to generate the list of players for the specific game. Use default & parameterized constructor

game/Indoor.java

```java
package game;

public class Indoor {
    String gameName; String[] players;
    public Indoor() {                                        // default constructor
        gameName = "Chess";
        players = new String[]{"Anand", "Kasparov"};
    }
    public Indoor(String g, String... p) { gameName = g; players = p; }   // parameterized
    public void display() {
        System.out.println("Indoor game: " + gameName + "  Players:");
        for (String p : players) System.out.println("  - " + p);
    }
}
```

game/Outdoor.java

```java
package game;

public class Outdoor {
    String gameName; String[] players;
    public Outdoor() {
        gameName = "Football";
        players = new String[]{"Messi", "Ronaldo", "Neymar"};
    }
    public Outdoor(String g, String... p) { gameName = g; players = p; }
    public void display() {
        System.out.println("Outdoor game: " + gameName + "  Players:");
        for (String p : players) System.out.println("  - " + p);
    }
}
```

GameDemo.java

```java
import game.*;

public class GameDemo {
    public static void main(String[] args) {
        new Indoor().display();                                  // default
        new Indoor("Carrom", "Amit", "Sumit").display();         // parameterized
        new Outdoor().display();
        new Outdoor("Cricket", "Virat", "Rohit", "Dhoni").display();
    }
}
```

Run: `javac game/*.java GameDemo.java`
`java GameDemo`

Output

```
Indoor game: Chess  Players:
  - Anand
  - Kasparov
Indoor game: Carrom  Players:
  - Amit
  - Sumit
Outdoor game: Football  Players:
  - Messi
  - Ronaldo
  - Neymar
Outdoor game: Cricket  Players:
  - Virat
  - Rohit
  - Dhoni
```

Q2 [15 marks]

Write a JavaScript program using an arrow function to calculate the total price of a product. Display the bill using template literals and string interpolation.

bill.js

```javascript
// Arrow function to calculate the total price
const totalPrice = (price, qty, gstPercent) => {
  const amount = price * qty;
  const gst = amount * gstPercent / 100;
  return { amount, gst, final: amount + gst };
};

const product = "Wireless Mouse", price = 750, qty = 3, gstRate = 18;
```

```
const bill = totalPrice(price, qty, gstRate);

console.log(`
------------- BILL -------------
Product  : ${product}
Price    : Rs. ${price}
Quantity : ${qty}
Amount   : Rs. ${bill.amount}
GST (${gstRate}%): Rs. ${bill.gst}
Total    : Rs. ${bill.final}
--------------------------------`);
```

Run: `node bill.js`

Output

```
------------- BILL -------------
Product  : Wireless Mouse
Price    : Rs. 750
Quantity : 3
Amount   : Rs. 2250
GST (18%): Rs. 405
Total    : Rs. 2655
--------------------------------
```

Q3 Viva [5 marks]

- **What is a default constructor?** Constructor without parameters; initialises fields with default values.
- **What are varargs (String... p)?** A method parameter that accepts zero or more arguments as an array.
- **What does toFixed(2) do?** Formats a number with 2 decimal places (returns a string).

Slip 13

Q1 [15 marks]

Define a “Clock” class that does the following; a. Accept Hours, Minutes and Seconds b. Check the validity of numbers c. Set the time to AM/PM mode Use the necessary constructors and methods to do the above task

ClockDemo.java

```java
import java.util.*;

class Clock {
    int h, m, s;
    boolean valid;

    Clock() { h = m = s = 0; valid = true; }
    Clock(int h, int m, int s) {
        this.h = h; this.m = m; this.s = s;
        valid = (h >= 0 && h < 24) && (m >= 0 && m < 60) && (s >= 0 && s < 60);   // validity check
    }

    void show() {
        if (!valid) { System.out.println("Invalid time!"); return; }
        String mode = (h >= 12) ? "PM" : "AM";                    // AM/PM mode
        int h12 = (h % 12 == 0) ? 12 : h % 12;
        System.out.printf("24-hour: %02d:%02d:%02d  ->  12-hour: %02d:%02d:%02d %s%n",
                          h, m, s, h12, m, s, mode);
    }
}

public class ClockDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter hours minutes seconds: ");
        new Clock(sc.nextInt(), sc.nextInt(), sc.nextInt()).show();
    }
}
```

Run: `javac ClockDemo.java`
`java ClockDemo`

Output

```
Enter hours minutes seconds: 15 45 30
24-hour: 15:45:30  ->  12-hour: 03:45:30 PM
```

Q2 [15 marks]

Write a JavaScript program to swap two numbers without using a third variable using array destructuring

swap.js

```javascript
let a = 10, b = 25;
console.log(`Before swap: a = ${a}, b = ${b}`);

[a, b] = [b, a];                    // array destructuring swaps without a third variable

console.log(`After swap : a = ${a}, b = ${b}`);
```

Run: `node swap.js`

Output

```
Before swap: a = 10, b = 25
After swap : a = 25, b = 10
```

Q3 Viva [5 marks]

- **How is 12-hour time obtained?** h % 12 (0 becomes 12); AM if h < 12 else PM.
- **What is constructor overloading?** More than one constructor with different parameter lists.
- **What is array destructuring?** [a, b] = [b, a] unpacks array values into variables; used here to swap.

Slip 14

Q1 [15 marks]

Write a JavaFX program that takes a numeric input from a user via a TextField and displays Whether it is a prime number in an output field upon clicking a “Process” button.

PrimeFX.java

```java
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class PrimeFX extends Application {
    @Override
    public void start(Stage stage) {
        TextField input = new TextField();
        input.setPromptText("Enter a number");
        Button process = new Button("Process");
        TextField output = new TextField();
        output.setEditable(false);

        process.setOnAction(e -> {                      // action listener
            try {
                int n = Integer.parseInt(input.getText().trim());
                boolean prime = n > 1;
                for (int i = 2; i * i <= n && prime; i++)
                    if (n % i == 0) prime = false;
                output.setText(n + (prime ? " is a Prime number" : " is NOT a Prime number"));
            } catch (NumberFormatException ex) {
                output.setText("Please enter a valid integer");
            }
        });

        VBox root = new VBox(10, new Label("Number:"), input, process, output);
        root.setPadding(new Insets(15));
        stage.setScene(new Scene(root, 300, 200));
        stage.setTitle("Prime Number Check");
        stage.show();
    }

    public static void main(String[] args) { launch(args); }
}
```

Run: `javac --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls PrimeFX.java`
`java --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls PrimeFX`

Output (GUI window)

A window with a TextField, a "Process" button and an output TextField. Enter 17 and click Process -> output shows "17 is a Prime number"; 18 -> "18 is NOT a Prime number".
JavaFX is not installed in the preparation environment, so this program was written and reviewed but not executed.

Q2 [15 marks]

Create an object containing employee details such as name, department, and salary. Use object destructuring to extract and display the values

employee_destructuring.js

```javascript
const employee = {
  name: "Amit Sharma",
  department: "IT",
  salary: 55000
};

// Object destructuring: variable names must match the property names
const { name, department, salary } = employee;

console.log("Name       :", name);
console.log("Department :", department);
console.log("Salary     :", salary);

// Extra: rename while destructuring and give a default value
const { name: empName, city = "Pune" } = employee;
console.log(empName, "works in", city);
```

Run: `node employee_destructuring.js`

Output

```
Name       : Amit Sharma
Department : IT
Salary     : 55000
Amit Sharma works in Pune
```

Q3 Viva [5 marks]

- **Structure of a JavaFX program?** Class extends Application; start(Stage) builds Scene with nodes; main calls launch().
- **How do you handle a button click?** button.setOnAction(e -> {...}) registers an ActionEvent handler.
- **Object destructuring rule?** Variable names must match property names: const {name, salary} = obj.

Slip 15

Q1 [15 marks]

*Write a JavaFX program to build a simple calculator with digit buttons (0-9) and basic arithmetic operator buttons (+, -, *, /). Implement action listeners for each button (digits and operators) to perform calculations.*

CalculatorFX.java

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.stage.Stage;

public class CalculatorFX extends Application {
    TextField display = new TextField("0");
    double first = 0; String op = ""; boolean startNew = true;

    @Override
    public void start(Stage stage) {
        display.setEditable(false);
        String[] keys = {"7","8","9","/", "4","5","6","*", "1","2","3","-", "0","C","=","+"};
        GridPane grid = new GridPane();
        grid.setHgap(5); grid.setVgap(5);
        for (int i = 0; i < keys.length; i++) {
            Button b = new Button(keys[i]);
            b.setPrefSize(55, 45);
            b.setOnAction(e -> press(b.getText()));      // one listener for every button
            grid.add(b, i % 4, i / 4);
        }
        VBox root = new VBox(10, display, grid);
        root.setPadding(new Insets(10));
        stage.setScene(new Scene(root));
        stage.setTitle("Calculator");
        stage.show();
    }

    void press(String k) {
        if (k.matches("[0-9]")) {                        // digit
            display.setText(startNew ? k : display.getText() + k);
            startNew = false;
        } else if (k.equals("C")) {                      // clear
            display.setText("0"); first = 0; op = ""; startNew = true;
        } else if (k.equals("=")) {                      // calculate
            if (!op.isEmpty()) display.setText(calc(first, Double.parseDouble(display.getText()), op));
            op = ""; startNew = true;
        } else {                                         // operator + - * /
            first = Double.parseDouble(display.getText());
            op = k; startNew = true;
        }
    }

    String calc(double a, double b, String op) {
        switch (op) {
            case "+": return String.valueOf(a + b);
            case "-": return String.valueOf(a - b);
            case "*": return String.valueOf(a * b);
            default:  return b == 0 ? "Error" : String.valueOf(a / b);
        }
    }

    public static void main(String[] args) { launch(args); }
}
```

Run: `javac --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls CalculatorFX.java`
`java --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls CalculatorFX`

Output (GUI window)

A calculator window: display field on top and a 4x4 grid of buttons (digits, + - * /, C, =). 7 + 8 = shows 15.0.
JavaFX is not installed in the preparation environment, so this program was written and reviewed but not executed.

Q2 [15 marks]

Write a JavaScript program to create a copy of an existing array using the spread operator and add a new element to the copied array.

spread_copy.js

```
const original = [10, 20, 30];
const copy = [...original];         // spread operator copies the elements
copy.push(40);                      // add a new element to the copy only

console.log("Original array:", original);
console.log("Copied array  :", copy);
```

```
console.log("Same array?    ", original === copy);   // false -> it is a real copy

const withMiddle = [...original, 99, ...copy];       // spread can also merge arrays
console.log("Merged        :", withMiddle);
```

Run: `node spread_copy.js`

Output

```
Original array: [ 10, 20, 30 ]
Copied array  : [ 10, 20, 30, 40 ]
Same array?     false
Merged        : [
  10, 20, 30, 99,
  10, 20, 30, 40
]
```

Q3 Viva [5 marks]

- **Which layouts are in JavaFX?** VBox, HBox, GridPane, BorderPane, StackPane, FlowPane.
- **What is the spread operator?** ... expands an array/object, [...a] makes a shallow copy.
- **Why is [...a] better than b = a?** b = a copies only the reference; spread creates a new array.

Slip 16

Q1 [15 marks]

Write a JavaFX application that simulates a shopping cart. Add items with buttons, and implement action listeners to update the cart when an item is added

CartFX.java

```java
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.stage.Stage;

public class CartFX extends Application {
    double total = 0;

    @Override
    public void start(Stage stage) {
        String[] names  = {"Laptop", "Mouse", "Keyboard", "Headphones"};
        double[] prices = {45000, 500, 1200, 2500};

        ListView<String> cart = new ListView<>();               // shows cart items
        Label totalLabel = new Label("Total: Rs. 0");

        HBox buttons = new HBox(10);
        for (int i = 0; i < names.length; i++) {
            String name = names[i]; double price = prices[i];
            Button b = new Button("Add " + name + " (" + price + ")");
            b.setOnAction(e -> {                                // update cart on click
                cart.getItems().add(name + "  -  Rs. " + price);
                total += price;
                totalLabel.setText("Total: Rs. " + total);
            });
            buttons.getChildren().add(b);
        }
        Button clear = new Button("Clear Cart");
        clear.setOnAction(e -> { cart.getItems().clear(); total = 0; totalLabel.setText("Total: Rs. 0"); });

        VBox root = new VBox(10, new Label("Shopping Cart"), buttons, cart, totalLabel, clear);
        root.setPadding(new Insets(15));
        stage.setScene(new Scene(root, 650, 350));
        stage.setTitle("Shopping Cart");
        stage.show();
    }

    public static void main(String[] args) { launch(args); }
}
```

Run: `javac --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls CartFX.java`
`java --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls CartFX`

Output (GUI window)

Buttons "Add Laptop / Mouse / Keyboard / Headphones"; each click appends the item to the cart list and updates the "Total" label. "Clear Cart" empties it.
JavaFX is not installed in the preparation environment, so this program was written and reviewed but not executed.

Q2 [15 marks]

Write a JavaScript program using a rest parameter to accept any number of values and calculate their sum

rest_sum.js

```javascript
// Rest parameter (...nums) collects any number of arguments into an array
function sum(...nums) {
  let total = 0;
  for (const n of nums) total += n;
  return total;
}

console.log("sum(1, 2, 3)          =", sum(1, 2, 3));
console.log("sum(10, 20, 30, 40)   =", sum(10, 20, 30, 40));
console.log("sum()                 =", sum());

const sum2 = (...nums) => nums.reduce((t, n) => t + n, 0);   // arrow version
console.log("sum2(5, 5, 5, 5, 5)   =", sum2(5, 5, 5, 5, 5));
```

Run: `node rest_sum.js`

Output

```
sum(1, 2, 3)          = 6
sum(10, 20, 30, 40)   = 100
sum()                 = 0
sum2(5, 5, 5, 5, 5)   = 25
```

Q3 Viva [5 marks]

- **Which control shows a list?** ListView<String>; add items with getItems().add().
- **Rest vs spread?** Rest (...nums) gathers arguments into an array; spread expands an array into elements.
- **Rule for rest parameter?** It must be the last parameter of the function.

Slip 17

Q1 [15 marks]

Write a JavaFX application that changes the background color of a window based on a combination of keys pressed (e.g., Ctrl+Alt+O changes the background to orange).

KeyColorFX.java

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.input.KeyCode;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class KeyColorFX extends Application {
    @Override
    public void start(Stage stage) {
        StackPane root = new StackPane(new Label("Ctrl+Alt+O = Orange, R = Red, G = Green, B = Blue"));
        Scene scene = new Scene(root, 450, 250);

        scene.setOnKeyPressed(e -> {
            if (e.isControlDown() && e.isAltDown()) {            // key combination
                KeyCode k = e.getCode();
                if (k == KeyCode.O)      root.setStyle("-fx-background-color: orange;");
                else if (k == KeyCode.R) root.setStyle("-fx-background-color: red;");
                else if (k == KeyCode.G) root.setStyle("-fx-background-color: green;");
                else if (k == KeyCode.B) root.setStyle("-fx-background-color: blue;");
            }
        });

        stage.setScene(scene);
        stage.setTitle("Key Combination Colors");
        stage.show();
    }

    public static void main(String[] args) { launch(args); }
}
```

Run: `javac --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls KeyColorFX.java`
`java --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls KeyColorFX`

Output (GUI window)

A blank window; pressing Ctrl+Alt+O turns the background orange (Ctrl+Alt+R red, G green, B blue).
JavaFX is not installed in the preparation environment, so this program was written and reviewed but not executed.

Q2 [15 marks]

Create a JavaScript module containing an arithmetic operation functions. Export the function and import it into another JavaScript file to perform all operations.

mathops.mjs

```javascript
// mathops.mjs  -> module that exports the functions
export const add      = (a, b) => a + b;
export const subtract = (a, b) => a - b;
export const multiply = (a, b) => a * b;
export const divide   = (a, b) => (b === 0 ? "Cannot divide by zero" : a / b);
```

main.mjs

```javascript
// main.mjs  -> imports the module and uses it   (run: node main.mjs)
import { add, subtract, multiply, divide } from "./mathops.mjs";

console.log("12 + 4 =", add(12, 4));
console.log("12 - 4 =", subtract(12, 4));
console.log("12 * 4 =", multiply(12, 4));
console.log("12 / 4 =", divide(12, 4));
console.log("12 / 0 =", divide(12, 0));
```

Run: `node main.mjs`

Output

```
12 + 4 = 16
12 - 4 = 8
12 * 4 = 48
12 / 4 = 3
12 / 0 = Cannot divide by zero
```

Q3 Viva [5 marks]

- **How to detect key combination in JavaFX?** scene.setOnKeyPressed(e -> e.isControlDown() && e.isAltDown() && e.getCode()==KeyCode.O).
- **Stage vs Scene?** Stage is the window; Scene is the content shown in it.

- **require vs import?** require is CommonJS; import/export is ES module syntax (.mjs or type: module).

Slip 18

Q1 [15 marks]

Creating an application with multiple buttons (e.g., Red, Green, Blue) that print their respective color or change the background color when clicked.

ColorButtonsFX.java

```java
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.stage.Stage;

public class ColorButtonsFX extends Application {
    @Override
    public void start(Stage stage) {
        Label msg = new Label("Click a button");
        Button red = new Button("Red"), green = new Button("Green"), blue = new Button("Blue");
        VBox root = new VBox(15, new HBox(10, red, green, blue), msg);
        root.setPadding(new Insets(20));

        red.setOnAction(e -> pick("Red", "red", root, msg));
        green.setOnAction(e -> pick("Green", "lightgreen", root, msg));
        blue.setOnAction(e -> pick("Blue", "lightblue", root, msg));

        stage.setScene(new Scene(root, 300, 150));
        stage.setTitle("Color Buttons");
        stage.show();
    }

    void pick(String name, String css, VBox root, Label msg) {
        System.out.println(name);                              // print the colour
        msg.setText("You clicked " + name);
        root.setStyle("-fx-background-color: " + css + ";");   // change background
    }

    public static void main(String[] args) { launch(args); }
}
```

Run: `javac --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls ColorButtonsFX.java`
`java --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls ColorButtonsFX`

Output (GUI window)

Three buttons Red, Green, Blue. Clicking one prints its colour in the console, shows "You clicked Red" and changes the window background.
JavaFX is not installed in the preparation environment, so this program was written and reviewed but not executed.

Q2 [15 marks]

Create a Node.js HTTP server that sends an appropriate response to a client request.

server.js

```javascript
const http = require("http");

const server = http.createServer((req, res) => {
  console.log(req.method, req.url);                      // log every request

  if (req.url === "/") {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end("<h1>Hello from Node.js server!</h1>");
  } else if (req.url === "/api") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify({ message: "This is JSON", status: "ok" }));
  } else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("404 - Page Not Found");
  }
});

server.listen(3000, () => console.log("Server running at http://localhost:3000/"));
```

Run: `node server.js (then open http://localhost:3000 in a browser)`

Output

```
--- Server console ---
Server running at http://localhost:3000/
GET /
GET /api
GET /contact

--- Browser / client side ---
$ curl http://localhost:3000/
```

```
HTTP/1.1 200 OK
<h1>Hello from Node.js server!</h1>
$ curl http://localhost:3000/api
HTTP/1.1 200 OK
{"message":"This is JSON","status":"ok"}
$ curl http://localhost:3000/contact
HTTP/1.1 404 Not Found
404 - Page Not Found
```

Q3 Viva [5 marks]

- **How to change background in JavaFX?** node.setStyle("-fx-background-color: red;") (CSS).
- **Which module creates a web server?** http - http.createServer((req,res)=>{...}).listen(port).
- **What are status codes 200 and 404?** 200 OK - success; 404 Not Found - page missing.

Slip 19

Q1 [15 marks]

Write a Java program to accept a file name from the user and count the number of characters, words, and lines present in the file.

FileCount.java

```java
import java.io.*;
import java.util.*;

public class FileCount {
    public static void main(String[] args) throws IOException {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter file name: ");
        File f = new File(sc.nextLine());

        int chars = 0, words = 0, lines = 0;
        try (BufferedReader br = new BufferedReader(new FileReader(f))) {
            String line;
            while ((line = br.readLine()) != null) {
                lines++;
                chars += line.length();
                if (!line.trim().isEmpty())
                    words += line.trim().split("\\s+").length;
            }
        }
        System.out.println("Characters = " + chars);
        System.out.println("Words      = " + words);
        System.out.println("Lines      = " + lines);
    }
}
```

Run: `javac FileCount.java`
`java FileCount` Create a text file (e.g. **data.txt**) first.

Output

```
Enter file name: data.txt
Characters = 36
Words      = 9
Lines      = 3
```

Q2 [15 marks]

Write a JavaScript program using async/await to simulate user login. If the username and password are correct, display a success message; otherwise, handle the error using try-catch

login.js

```javascript
const USER = "admin", PASS = "1234";

// Returns a Promise that resolves/rejects after 1 second (simulates a server call)
const checkLogin = (username, password) =>
  new Promise((resolve, reject) => {
    setTimeout(() => {
      if (username === USER && password === PASS) resolve("Login successful. Welcome " + username + "!");
      else reject(new Error("Invalid username or password"));
    }, 1000);
  });

async function login(username, password) {
  try {
    const msg = await checkLogin(username, password);     // wait for the Promise
    console.log(msg);
  } catch (err) {
    console.log("Login failed:", err.message);            // error handled here
  }
}

(async () => {
  await login("admin", "1234");      // correct
  await login("admin", "abcd");      // wrong
})();
```

Run: `node login.js`

Output

```
Login successful. Welcome admin!
Login failed: Invalid username or password
```

Q3 Viva [5 marks]

- **How to count words?** trim the line and split on "\\s+"; count the array length.
- **What does async/await do?** async function returns a Promise; await pauses until it settles, making async code look sequential.

- **Promise states?** pending, fulfilled (resolved), rejected.

Slip 20

Q1 [15 marks]

Design a screen to handle the Mouse Events such as MOUSE_MOVED and MOUSE_CLICK and display the position of the Mouse_Click in a TextField.

MouseFX.java

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.input.MouseEvent;
import javafx.scene.layout.*;
import javafx.stage.Stage;

public class MouseFX extends Application {
    @Override
    public void start(Stage stage) {
        TextField clickPos = new TextField();                   // shows click position
        clickPos.setEditable(false);
        Label moveLabel = new Label("Move the mouse over the pane");

        Pane pane = new Pane();
        pane.setPrefSize(350, 200);
        pane.setStyle("-fx-background-color: lightyellow;");

        pane.setOnMouseMoved(e ->                               // MOUSE_MOVED
            moveLabel.setText("Moved: x=" + (int) e.getX() + ", y=" + (int) e.getY()));
        pane.addEventHandler(MouseEvent.MOUSE_CLICKED, e ->     // MOUSE_CLICKED
            clickPos.setText("Clicked at x=" + (int) e.getX() + ", y=" + (int) e.getY()));

        VBox root = new VBox(8, pane, moveLabel, new Label("Click position:"), clickPos);
        stage.setScene(new Scene(root));
        stage.setTitle("Mouse Events");
        stage.show();
    }

    public static void main(String[] args) { launch(args); }
}
```

Run: `javac --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls MouseFX.java`
`java --module-path "path\to\javafx-sdk\lib" --add-modules javafx.controls MouseFX`

Output (GUI window)

A yellow pane: moving the mouse shows live x,y in a label; clicking shows "Clicked at x=.., y=.." in the TextField.
JavaFX is not installed in the preparation environment, so this program was written and reviewed but not executed.

Q2 [15 marks]

Write a JavaScript program using a Promise to divide two numbers. If the denominator is zero, reject the Promise and handle the error.

divide_promise.js

```
const divide = (a, b) =>
  new Promise((resolve, reject) => {
    if (b === 0) reject("Error: denominator cannot be zero");
    else resolve(a / b);
  });

divide(20, 4)
  .then(r => console.log("20 / 4 =", r))
  .catch(err => console.log(err));

divide(10, 0)
  .then(r => console.log("10 / 0 =", r))
  .catch(err => console.log(err))                // reject -> catch
  .finally(() => console.log("Division attempts finished"));
```

Run: `node divide_promise.js`

Output

```
20 / 4 = 5
Error: denominator cannot be zero
Division attempts finished
```

Q3 Viva [5 marks]

- **Which events are used?** MouseEvent.MOUSE_MOVED and MOUSE_CLICKED; e.getX()/e.getY() give position.
- **then/catch/finally?** then handles success, catch handles rejection, finally always runs.
- **Why Promises?** Avoid callback hell and handle async results/errors cleanly.

Slip 21

Q1 [15 marks]

Write a program to create parent class College (cno, cname, caddr) and derived class Department (dno, dname) from College. Write a necessary methods to display College details.

CollegeDemo.java

```java
class College {
    int cno; String cname, caddr;
    College(int cno, String cname, String caddr) {
        this.cno = cno; this.cname = cname; this.caddr = caddr;
    }
    void displayCollege() {
        System.out.println("College No: " + cno + ", Name: " + cname + ", Address: " + caddr);
    }
}

class Department extends College {
    int dno; String dname;
    Department(int cno, String cname, String caddr, int dno, String dname) {
        super(cno, cname, caddr);
        this.dno = dno; this.dname = dname;
    }
    void displayAll() {
        displayCollege();
        System.out.println("Department No: " + dno + ", Name: " + dname);
    }
}

public class CollegeDemo {
    public static void main(String[] args) {
        new Department(101, "ABC College", "Pune", 5, "Computer Science").displayAll();
    }
}
```

Run: `javac CollegeDemo.java`
`java CollegeDemo`

Output

```
College No: 101, Name: ABC College, Address: Pune
Department No: 5, Name: Computer Science
```

Q2 [15 marks]

Write a Node.js program using the path module to perform operations such as joining, resolving and extracting file/directory information.

path_demo.js

```javascript
const path = require("path");

const p = "/home/student/projects/app/index.js";

console.log("join     :", path.join("projects", "app", "..", "docs", "a.txt"));
console.log("resolve  :", path.resolve("docs", "a.txt"));        // absolute path from cwd
console.log("basename :", path.basename(p));                     // index.js
console.log("basename (no ext):", path.basename(p, ".js"));      // index
console.log("dirname  :", path.dirname(p));
console.log("extname  :", path.extname(p));
console.log("isAbsolute:", path.isAbsolute(p), path.isAbsolute("docs/a.txt"));
console.log("normalize:", path.normalize("/home//student/../student/./app"));
console.log("relative :", path.relative("/home/student/projects", p));
console.log("parse    :", path.parse(p));
console.log("format   :", path.format({ dir: "/data", name: "report", ext: ".pdf" }));
```

Run: `node path_demo.js`

Output

```
join     : projects/docs/a.txt
resolve  : /home/student/project/docs/a.txt
basename : index.js
basename (no ext): index
dirname  : /home/student/projects/app
extname  : .js
isAbsolute: true false
normalize: /home/student/app
relative : app/index.js
parse    : {
  root: '/',
  dir: '/home/student/projects/app',
  base: 'index.js',
  ext: '.js',
  name: 'index'
}
format   : /data/report.pdf
```

Q3 Viva [5 marks]

- **Why super(...) in Department?** To initialise the inherited College members; must be the first statement.
- **path.join vs path.resolve?** join concatenates segments; resolve returns an absolute path from the current directory.
- **What is __dirname?** Absolute path of the directory containing the current script.

Slip 22

Q1 [15 marks]

Write a program using interface, create a class Product (product_id, product_name, product_cost, product_quantity) default and parameterized constructor. Create objects of class product and display the contents of each object and Also display the object count.

ProductDemo.java

```
interface Showable {
    void display();
}

class Product implements Showable {
    int product_id, product_quantity; String product_name; double product_cost;
    static int count = 0;                                    // counts objects

    Product() { this(0, "None", 0, 0); }                     // default constructor
    Product(int id, String name, double cost, int qty) {
        product_id = id; product_name = name; product_cost = cost; product_quantity = qty;
        count++;
    }
    public void display() {
        System.out.println(product_id + " | " + product_name + " | Rs." + product_cost + " | Qty " + product_quantity);
    }
}

public class ProductDemo {
    public static void main(String[] args) {
        Product[] p = { new Product(), new Product(1, "Pen", 10, 100), new Product(2, "Notebook", 50, 40) };
        for (Product x : p) x.display();
        System.out.println("Total objects created = " + Product.count);
    }
}
```

Run: `javac ProductDemo.java`
`java ProductDemo`

Output

```
0 | None | Rs.0.0 | Qty 0
1 | Pen | Rs.10.0 | Qty 100
2 | Notebook | Rs.50.0 | Qty 40
Total objects created = 3
```

Q2 [15 marks]

Write a Node.js program using the fs module to perform file operations asynchronously.

fs_async.js

```
const fs = require("fs");

// Callback style (asynchronous): the callback runs when the operation is finished
fs.writeFile("demo.txt", "Hello from Node.js\n", (err) => {
  if (err) return console.log("Write error:", err.message);
  console.log("1. File written");

  fs.appendFile("demo.txt", "Second line appended\n", (err) => {
    if (err) return console.log("Append error:", err.message);
    console.log("2. Data appended");

    fs.readFile("demo.txt", "utf8", (err, data) => {
      if (err) return console.log("Read error:", err.message);
      console.log("3. File content:\n" + data);

      fs.unlink("demo.txt", (err) => {
        console.log(err ? "Delete error: " + err.message : "4. File deleted");
      });
    });
  });
});

console.log("This line prints first because the file operations are asynchronous");
```

Run: `node fs_async.js`

Output

```
This line prints first because the file operations are asynchronous
1. File written
2. Data appended
3. File content:
Hello from Node.js
Second line appended

4. File deleted
```

Q3 Viva [5 marks]

- **Interface vs abstract class?** Interface: only abstract methods (and constants) - a contract; abstract class may have state and concrete methods.
- **Why static count?** A static variable is shared by all objects, so it counts objects created.
- **Error-first callback?** Callback whose first parameter is err: (err, data) => {...}.

Slip 23

Q1 [15 marks]

Write a program for multilevel inheritance such that Country is inherited from Continent. State is inherited from Country. Display the place, State, Country and Continent.

MultilevelDemo.java

```
class Continent {
    String continent;
    Continent(String c) { continent = c; }
}

class Country extends Continent {
    String country;
    Country(String c, String co) { super(c); country = co; }
}

class State extends Country {
    String state, place;
    State(String c, String co, String s, String p) { super(c, co); state = s; place = p; }
    void display() {
        System.out.println("Place    : " + place);
        System.out.println("State    : " + state);
        System.out.println("Country  : " + country);
        System.out.println("Continent: " + continent);
    }
}

public class MultilevelDemo {
    public static void main(String[] args) {
        new State("Asia", "India", "Maharashtra", "Pune").display();
    }
}
```

Run: `javac MultilevelDemo.java`
`java MultilevelDemo`

Output

```
Place    : Pune
State    : Maharashtra
Country  : India
Continent: Asia
```

Q2 [15 marks]

Write a Node.js program to create, write, read and append data to a file using the fs module.

fs_crud.js

```
const fs = require("fs");
const file = "student.txt";

fs.writeFileSync(file, "Name: Riya\n");                // create + write (overwrites)
console.log("File created and written");

fs.appendFileSync(file, "Class: TYBSc\n");             // append
console.log("Data appended");

const data = fs.readFileSync(file, "utf8");            // read
console.log("---- File content ----\n" + data + "----------------------");

console.log("File exists?", fs.existsSync(file));
```

Run: `node fs_crud.js`

Output

```
File created and written
Data appended
---- File content ----
Name: Riya
Class: TYBSc
----------------------
File exists? true
```

Q3 Viva [5 marks]

- **What is multilevel inheritance?** A chain: Continent -> Country -> State, each class extends the previous one.
- **writeFile vs appendFile?** writeFile overwrites the file; appendFile adds to the end.
- **Why pass "utf8" to readFile?** To get a string instead of a Buffer.

Slip 24

Q1 [15 marks]

Create an abstract class Shape with methods area & volume. Derive a class Cylinder (radius, height). Calculate area and volume

CylinderDemo.java

```java
abstract class Shape {
    abstract double area();
    abstract double volume();
}

class Cylinder extends Shape {
    double radius, height;
    Cylinder(double r, double h) { radius = r; height = h; }
    double area()   { return 2 * Math.PI * radius * (radius + height); }   // total surface area
    double volume() { return Math.PI * radius * radius * height; }
}

public class CylinderDemo {
    public static void main(String[] args) {
        Cylinder c = new Cylinder(7, 10);
        System.out.printf("Area   = %.2f%n", c.area());
        System.out.printf("Volume = %.2f%n", c.volume());
    }
}
```

Run: `javac CylinderDemo.java`
`java CylinderDemo`

Output

```
Area   = 747.70
Volume = 1539.38
```

Q2 [15 marks]

Write a Node.js program to demonstrate synchronous and asynchronous file operations and compare their execution.

sync_vs_async.js

```javascript
const fs = require("fs");
fs.writeFileSync("big.txt", "Node.js file handling\n".repeat(200000));   // make a sample file

// ---- Synchronous: blocks the program until the file is read ----
console.log("SYNC  start");
let t1 = process.hrtime.bigint();
const d1 = fs.readFileSync("big.txt", "utf8");
let t2 = process.hrtime.bigint();
console.log("SYNC  done, length =", d1.length, "time =", Number(t2 - t1) / 1e6, "ms");
console.log("SYNC  next statement runs only AFTER reading finished");

// ---- Asynchronous: program continues, callback runs later ----
console.log("ASYNC start");
t1 = process.hrtime.bigint();
fs.readFile("big.txt", "utf8", (err, d2) => {
  t2 = process.hrtime.bigint();
  console.log("ASYNC done, length =", d2.length, "time =", Number(t2 - t1) / 1e6, "ms");
  fs.unlinkSync("big.txt");
});
console.log("ASYNC next statement runs immediately (before the file is read)");
```

Run: `node sync_vs_async.js`

Output

```
SYNC  start
SYNC  done, length = 4400000 time = 16.054494 ms
SYNC  next statement runs only AFTER reading finished
ASYNC start
ASYNC next statement runs immediately (before the file is read)
ASYNC done, length = 4400000 time = 9.367962 ms
```

Q3 Viva [5 marks]

- **Formulas used?** Area = 2*PI*r*(r+h), Volume = PI*r^2*h.
- **Can an abstract class have constructors?** Yes, and also concrete methods; it just cannot be instantiated.
- **Sync vs async?** Sync blocks the program until finished; async continues and runs a callback later (event loop).

Slip 25

Q1 [15 marks]

Write a Java program that defines an interface Calculator containing methods add() and subtract() and implements it in class SimpleCalc.

CalcDemo.java

```java
interface Calculator {
    int add(int a, int b);
    int subtract(int a, int b);
}

class SimpleCalc implements Calculator {
    public int add(int a, int b)      { return a + b; }
    public int subtract(int a, int b) { return a - b; }
}

public class CalcDemo {
    public static void main(String[] args) {
        Calculator c = new SimpleCalc();
        System.out.println("Addition    : " + c.add(20, 10));
        System.out.println("Subtraction : " + c.subtract(20, 10));
    }
}
```

Run: `javac CalcDemo.java`
`java CalcDemo`

Output

```
Addition    : 30
Subtraction : 10
```

Q2 [15 marks]

Write a Node.js program to perform directory management operations such as creating, listing, renaming and deleting directories. Also demonstrate JSON parsing and writing structured JSON data.

dir_json.js

```javascript
const fs = require("fs");

// ---- Directory management ----
fs.mkdirSync("mydir");                                  // create
fs.writeFileSync("mydir/a.txt", "file A");
fs.writeFileSync("mydir/b.txt", "file B");
console.log("Created :", fs.readdirSync("mydir"));      // list

fs.renameSync("mydir", "newdir");                       // rename
console.log("Renamed : newdir ->", fs.readdirSync("newdir"));

// ---- JSON writing and parsing ----
const student = { id: 1, name: "Riya", subjects: ["Java", "Web Tech"], passed: true };
fs.writeFileSync("newdir/student.json", JSON.stringify(student, null, 2));   // write JSON
const parsed = JSON.parse(fs.readFileSync("newdir/student.json", "utf8"));   // parse JSON
console.log("Parsed JSON:", parsed);
console.log("Name:", parsed.name, "| First subject:", parsed.subjects[0]);

fs.rmSync("newdir", { recursive: true });               // delete directory with its files
console.log("Deleted : newdir exists?", fs.existsSync("newdir"));
```

Run: `node dir_json.js`

Output

```
Created : [ 'a.txt', 'b.txt' ]
Renamed : newdir -> [ 'a.txt', 'b.txt' ]
Parsed JSON: { id: 1, name: 'Riya', subjects: [ 'Java', 'Web Tech' ], passed: true }
Name: Riya | First subject: Java
Deleted : newdir exists? false
```

Q3 Viva [5 marks]

- **Methods in an interface?** Implicitly public abstract; implementing class must define them all.
- **Directory functions in fs?** mkdirSync, readdirSync, renameSync, rmSync.
- **JSON.parse vs JSON.stringify?** parse converts JSON text to object; stringify converts object to JSON text.

Slip 26

Q1 [15 marks]

Define a class Student with data members student_name, student_rollno, total_lectures, attended_lectures. Create an object of Student. If attendance percentage is less than 75%, throw a user-defined exception "Student is Not Eligible for Exam", otherwise display student details.

StudentDemo.java

```java
import java.util.*;

class NotEligibleException extends Exception {
    NotEligibleException(String msg) { super(msg); }
}

class Student {
    String student_name; int student_rollno, total_lectures, attended_lectures;
    Student(String n, int r, int t, int a) {
        student_name = n; student_rollno = r; total_lectures = t; attended_lectures = a;
    }
    void check() throws NotEligibleException {
        double percent = attended_lectures * 100.0 / total_lectures;
        if (percent < 75) throw new NotEligibleException("Student is Not Eligible for Exam");
        System.out.println("Name: " + student_name + ", Roll No: " + student_rollno +
                           ", Attendance: " + percent + "%  -> Eligible");
    }
}

public class StudentDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Name, roll no, total lectures, attended: ");
        Student s = new Student(sc.next(), sc.nextInt(), sc.nextInt(), sc.nextInt());
        try {
            s.check();
        } catch (NotEligibleException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

Run: `javac StudentDemo.java`
`java StudentDemo`

Output

```
Name, roll no, total lectures, attended: Riya 12 100 70
Exception: Student is Not Eligible for Exam
```

Q2 [15 marks]

Write a Node.js program using the fs module to copy, rename and delete files asynchronously. Display appropriate success and error messages.

copy_rename_delete.js

```javascript
const fs = require("fs").promises;      // promise based (async/await) fs

async function main() {
  try {
    await fs.writeFile("source.txt", "Original content");
    await fs.copyFile("source.txt", "copy.txt");
    console.log("Success: source.txt copied to copy.txt");

    await fs.rename("copy.txt", "renamed.txt");
    console.log("Success: copy.txt renamed to renamed.txt");

    await fs.unlink("renamed.txt");
    console.log("Success: renamed.txt deleted");

    await fs.unlink("source.txt");
    console.log("Success: source.txt deleted");

    await fs.unlink("missing.txt");                    // does not exist -> error
  } catch (err) {
    console.log("Error:", err.code, "-", err.message);
  }
}
main();
```

Run: `node copy_rename_delete.js`

Output

```
Success: source.txt copied to copy.txt
Success: copy.txt renamed to renamed.txt
Success: renamed.txt deleted
```

```
Success: source.txt deleted
Error: ENOENT - ENOENT: no such file or directory, unlink 'missing.txt'
```

Q3 Viva [5 marks]

- **Attendance formula?** attended * 100.0 / total; below 75 -> throw exception.
- **fs functions for files?** copyFile, rename, unlink (delete).
- **What is ENOENT?** Error code: no such file or directory.

Slip 27

Q1 [15 marks]

Write a program to define class Person with data members as Personname,Aadharno, Panno. Accept information for 5 objects and display appropriate information (use this keyword).

PersonDemo.java

```java
import java.util.*;

class Person {
    String personname, aadharno, panno;
    Person(String personname, String aadharno, String panno) {
        this.personname = personname;                        // 'this' separates field from parameter
        this.aadharno = aadharno;
        this.panno = panno;
    }
    void display() {
        System.out.println(this.personname + " | Aadhar: " + this.aadharno + " | PAN: " + this.panno);
    }
}

public class PersonDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Person[] p = new Person[5];
        for (int i = 0; i < 5; i++) {
            System.out.print("Person " + (i + 1) + " (name aadhar pan): ");
            p[i] = new Person(sc.next(), sc.next(), sc.next());
        }
        System.out.println("\n--- Person details ---");
        for (Person x : p) x.display();
    }
}
```

Run: `javac PersonDemo.java`
`java PersonDemo`

Output

```
Person 1 (name aadhar pan): Amit 1111 ABCDE1
Person 2 (name aadhar pan): Sara 2222 FGHIJ2
Person 3 (name aadhar pan): Ravi 3333 KLMNO3
Person 4 (name aadhar pan): Neha 4444 PQRST4
Person 5 (name aadhar pan): Om 5555 UVWXY5

--- Person details ---
Amit | Aadhar: 1111 | PAN: ABCDE1
Sara | Aadhar: 2222 | PAN: FGHIJ2
Ravi | Aadhar: 3333 | PAN: KLMNO3
Neha | Aadhar: 4444 | PAN: PQRST4
Om | Aadhar: 5555 | PAN: UVWXY5
```

Q2 [15 marks]

Write a Node.js program to read a JSON file, modify selected properties of the JSON object, and write the updated JSON data back to the file using the fs module

json_update.js

```javascript
const fs = require("fs");

// create a sample JSON file first
fs.writeFileSync("employee.json", JSON.stringify({ id: 101, name: "Amit", salary: 40000, city: "Pune" }, null, 2));

const emp = JSON.parse(fs.readFileSync("employee.json", "utf8"));    // 1. read
console.log("Before:", emp);

emp.salary = 50000;                                                  // 2. modify
emp.city = "Mumbai";
emp.designation = "Developer";                                       //    add new property

fs.writeFileSync("employee.json", JSON.stringify(emp, null, 2));     // 3. write back
console.log("After :", JSON.parse(fs.readFileSync("employee.json", "utf8")));
```

Run: `node json_update.js`

Output

```
Before: { id: 101, name: 'Amit', salary: 40000, city: 'Pune' }
After : {
  id: 101,
  name: 'Amit',
  salary: 50000,
  city: 'Mumbai',
  designation: 'Developer'
}
```

Q3 Viva [5 marks]

- **Use of this keyword?** Refers to the current object; separates fields from parameters of the same name.
- **JSON.stringify(obj, null, 2)?** Converts to JSON text with 2-space indentation.
- **Steps to update a JSON file?** Read -> JSON.parse -> modify -> JSON.stringify -> write back.

Slip 28

Q1 [15 marks]

Define a class MyDate(Day, Month, year) with methods to accept and display a MyDateobject.Accept date as dd,mm,yyyy. Throw user defined exception "InvalidDateException" if the date is invalid.

DateDemo.java

```java
import java.util.*;

class InvalidDateException extends Exception {
    InvalidDateException(String msg) { super(msg); }
}

class MyDate {
    int day, month, year;
    void accept(int d, int m, int y) throws InvalidDateException {
        int[] days = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        boolean leap = (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
        if (leap) days[1] = 29;                                  // February in leap year
        if (y < 1 || m < 1 || m > 12 || d < 1 || d > days[m - 1])
            throw new InvalidDateException("InvalidDateException: " + d + "/" + m + "/" + y + " is not a valid date");
        day = d; month = m; year = y;
    }
    void display() { System.out.printf("Date: %02d/%02d/%d%n", day, month, year); }
}

public class DateDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter date as dd mm yyyy: ");
        MyDate d = new MyDate();
        try {
            d.accept(sc.nextInt(), sc.nextInt(), sc.nextInt());
            d.display();
        } catch (InvalidDateException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Run: `javac DateDemo.java`
`java DateDemo`

Output

```
Enter date as dd mm yyyy: 31 4 2024
InvalidDateException: 31/4/2024 is not a valid date
```

Q2 [15 marks]

Create a Node.js program to list files in a specified directory, filter files based on their extensions (such as .txt, .json and .js), and display the matching file names.

list_filter.js

```javascript
const fs = require("fs");
const path = require("path");

const dir = process.argv[2] || ".";                              // directory (default: current)
const exts = process.argv.slice(3).length ? process.argv.slice(3) : [".txt", ".json", ".js"];

const files = fs.readdirSync(dir);
console.log("All files in '" + dir + "':", files);

const matched = files.filter(f => exts.includes(path.extname(f)));   // filter by extension
console.log("Files with extensions " + exts.join(", ") + ":");
matched.forEach(f => console.log("  " + f));
console.log("Total matching files:", matched.length);
```

Run: `node list_filter.js files (optional: extra args = extensions, e.g. .txt .json)` Create a folder **files** containing a few .txt/.json/.js files first.

Output

```
All files in 'files': [
  'app.js',
  'data.json',
  'notes.txt',
  'photo.png',
  'readme.md',
  'style.css',
  'todo.txt'
]
Files with extensions .txt, .json, .js:
  app.js
  data.json
  notes.txt
```

```
  todo.txt
Total matching files: 4
```

Q3 Viva [5 marks]

- **Leap year rule?** Divisible by 4 and not by 100, or divisible by 400.
- **How to filter by extension?** fs.readdirSync(dir).filter(f => path.extname(f) === ".txt").
- **What is process.argv?** Array of command-line arguments; index 2 onwards are user arguments.

Slip 29

Q1 [15 marks]

Write a program to accept a number from the user, if number is zero then throw user defined exception “Number is 0” otherwise check whether no is prime or not (Use static keyword)

PrimeStatic.java

```java
import java.util.*;

class ZeroException extends Exception {
    ZeroException(String msg) { super(msg); }
}

public class PrimeStatic {
    static boolean isPrime(int n) {                              // static method
        if (n < 2) return false;
        for (int i = 2; i * i <= n; i++)
            if (n % i == 0) return false;
        return true;
    }

    static void check(int n) throws ZeroException {
        if (n == 0) throw new ZeroException("Number is 0");
        System.out.println(n + (isPrime(n) ? " is a prime number" : " is not a prime number"));
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        try {
            check(sc.nextInt());
        } catch (ZeroException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}
```

Run: `javac PrimeStatic.java`
`java PrimeStatic`

Output

```
Enter a number: 17
17 is a prime number
```

Q2 [15 marks]

Write a Node.js program to demonstrate file statistics using fs.stat(), including file size, file type, creation/access/modification information, and whether the path represents a file or directory.

file_stat.js

```javascript
const fs = require("fs").promises;

async function showStat(p) {
  try {
    const s = await fs.stat(p);
    console.log("\nPath         :", p);
    console.log("Size (bytes) :", s.size);
    console.log("Is file?     :", s.isFile());
    console.log("Is directory?:", s.isDirectory());
    console.log("Created      :", s.birthtime.toISOString());
    console.log("Accessed     :", s.atime.toISOString());
    console.log("Modified     :", s.mtime.toISOString());
  } catch (err) {
    console.log("\nError:", err.message);
  }
}

(async () => {
  await fs.writeFile("sample.txt", "Hello Node.js stat demo");
  await showStat("sample.txt");      // a file
  await showStat(".");               // a directory
  await showStat("nofile.txt");      // error case
})();
```

Run: `node file_stat.js`

Output

```
Path         : sample.txt
Size (bytes) : 23
Is file?     : true
Is directory?: false
Created      : 2026-10-11T05:57:46.089Z
Accessed     : 2026-10-11T05:57:46.089Z
Modified     : 2026-10-11T05:57:46.089Z
```

```
Path         : .
Size (bytes) : 4096
Is file?     : false
Is directory?: true
Created      : 2026-10-11T05:57:46.057Z
Accessed     : 2026-10-11T05:57:46.057Z
Modified     : 2026-10-11T05:57:46.089Z

Error: ENOENT: no such file or directory, stat 'nofile.txt'
```

Q3 Viva [5 marks]

- **Why static?** A static method belongs to the class and is called without an object.
- **Prime check logic?** Test divisors from 2 up to sqrt(n); 0 and 1 are not prime.
- **What does fs.stat give?** Stats object: size, isFile(), isDirectory(), birthtime, atime, mtime.

Slip 30

Q1 [15 marks]

Define a class MyNumber having one private integer data member. Write a default constructor initialize it to 0 and another constructor to initialize it to a value. Write methods isNegative, isPositive, isOdd, isEven. Use command line argument to pass a value to the object and perform the above operations.

MyNumber.java

```java
public class MyNumber {
    private int n;
    MyNumber()      { n = 0; }                                  // default constructor
    MyNumber(int n) { this.n = n; }

    boolean isNegative() { return n < 0; }
    boolean isPositive() { return n > 0; }
    boolean isOdd()      { return n % 2 != 0; }
    boolean isEven()     { return n % 2 == 0; }

    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Usage: java MyNumber <integer>");
            return;
        }
        MyNumber m = new MyNumber(Integer.parseInt(args[0]));    // command line argument
        System.out.println("Negative? " + m.isNegative());
        System.out.println("Positive? " + m.isPositive());
        System.out.println("Odd?      " + m.isOdd());
        System.out.println("Even?     " + m.isEven());
    }
}
```

Run: `javac MyNumber.java`
`java MyNumber -7`

Output

```
$ java MyNumber -7
Negative? true
Positive? false
Odd?      true
Even?     false
```

Q2 [15 marks]

Create a Node.js mini file-management application using the fs and path modules that allows the user to create, read, update, rename and delete text files through a menu-driven interface.

file_manager.js

```javascript
const fs = require("fs");
const path = require("path");
const readline = require("readline/promises");

const dir = path.join(__dirname, "files");              // all files live in ./files
fs.mkdirSync(dir, { recursive: true });
const fp = name => path.join(dir, name);

async function main() {
  const rl = readline.createInterface({ input: process.stdin, output: process.stdout });
  let choice = "";
  while (choice !== "7") {
    console.log("\n1.Create 2.Read 3.Update(append) 4.Rename 5.Delete 6.List 7.Exit");
    choice = (await rl.question("Choice: ")).trim();
    try {
      if (choice === "1") {
        const n = await rl.question("File name: ");
        fs.writeFileSync(fp(n), await rl.question("Content: ") + "\n");
        console.log("File created");
      } else if (choice === "2") {
        console.log(fs.readFileSync(fp(await rl.question("File name: ")), "utf8"));
      } else if (choice === "3") {
        const n = await rl.question("File name: ");
        fs.appendFileSync(fp(n), await rl.question("Text to add: ") + "\n");
        console.log("File updated");
      } else if (choice === "4") {
        const oldN = await rl.question("Old name: ");
        fs.renameSync(fp(oldN), fp(await rl.question("New name: ")));
        console.log("File renamed");
      } else if (choice === "5") {
        fs.unlinkSync(fp(await rl.question("File name: ")));
        console.log("File deleted");
      } else if (choice === "6") {
        console.log(fs.readdirSync(dir));
      }
    } catch (e) {
      console.log("Error:", e.message);
```

```
    }
  }
  rl.close();
}
main();
```

Run: `node file_manager.js`

Output

```
1.Create 2.Read 3.Update(append) 4.Rename 5.Delete 6.List 7.Exit
Choice: 1
File name: hello.txt
Content: First line
File created

1.Create 2.Read 3.Update(append) 4.Rename 5.Delete 6.List 7.Exit
Choice: 2
File name: hello.txt
First line


1.Create 2.Read 3.Update(append) 4.Rename 5.Delete 6.List 7.Exit
Choice: 3
File name: hello.txt
Text to add: Second line
File updated

1.Create 2.Read 3.Update(append) 4.Rename 5.Delete 6.List 7.Exit
Choice: 2
File name: hello.txt
First line
Second line


1.Create 2.Read 3.Update(append) 4.Rename 5.Delete 6.List 7.Exit
Choice: 4
Old name: hello.txt
New name: notes.txt
File renamed

1.Create 2.Read 3.Update(append) 4.Rename 5.Delete 6.List 7.Exit
Choice: 6
[ 'notes.txt' ]

1.Create 2.Read 3.Update(append) 4.Rename 5.Delete 6.List 7.Exit
Choice: 5
File name: notes.txt
File deleted

1.Create 2.Read 3.Update(append) 4.Rename 5.Delete 6.List 7.Exit
Choice: 2
File name: notes.txt
Error: ENOENT: no such file or directory, open '/home/student/project/files/notes.txt'

1.Create 2.Read 3.Update(append) 4.Rename 5.Delete 6.List 7.Exit
Choice: 7
```

Q3 Viva [5 marks]

- **How to read command line argument?** args[0] in main(String[] args); convert with Integer.parseInt.
- **Why private data member?** Encapsulation: data accessible only through the class methods.
- **Which module reads user input in Node?** readline (rl.question) - used here for the menu.